АЛГОРИТМЫН ҮНДЭС · PYTHON · ЭРЭМБЭЛЭЛТ 3/3

Merge sort

Нийлүүлэн эрэмбэлэлт: хуваж, эрэмбэлж, нийлүүлэх



Түвшин: Ойлголт + хэрэгжүүлэлт | **Жишээ:** Python 3 | **Урьдчилсан мэдлэг:** Insertion sort, функц, рекурс, Big O

Энэ хичээлийг уншсаны дараа та:

- Эрэмбэлэгдсэн хоёр жагсаалтыг **нийлүүлэх** (merge) функцийг өөрөө бичнэ.
- «Хуваж ял» (divide and conquer) санааг ойлгож, merge sort-ийг **рекурсээр** бичнэ.
- Рекурсын модыг зурж, кодоо гараар мөрдөнө.
- Яагаад **O(n log n)** болохыг тайлбарлаж, O(n²)-тэй бодит хэмжилтээр харьцуулна.
- Python-ий `sorted()`-ийг `key=`, `reverse=`-тэй зөв хэрэглэнэ.

Агуулга

01 Санаа: хуваж ял
02 Нийлүүлэх (merge)
03 Merge sort
04 Рекурсын мод
05 Яагаад O(n log n) вэ?
06 Түгээмэл алдаа ба тест
07 Гурван алгоритмыг хэмжих
08 Python-ий sorted()
09 Дасгал
10 Хураангуй

Нэг өгүүлбэрээр

Merge sort (нийлүүлэн эрэмбэлэлт): жагсаалтыг хоёр хуваа, хагас бүрийг ижил аргаар эрэмбэл, дараа нь эрэмбэлэгдсэн хоёр хагасыг нэг болгон **нийлүүл**.

01 Санаа: хувааж ял

> **Өмнөх хичээлүүдээс**
> Selection, insertion sort хоёулаа $O(n^2)$: $n = 1\,000\,000$ бол $\approx 5 \cdot 10^{11}$ харьцуулалт. Binary search хагасаар хуваах замаар $O(n)$-ийг $O(\log n)$ болгосныг санаарай (Хайлт 2/2). Merge sort ч мөн хуваах санааг ашиглана.

Зүйрлэл: багш 32 сурагчийн шалгалтын хуудсыг нэрээр нь эрэмбэлэх хэрэгтэй. Хуудсыг хоёр туслахдаа хувааж өгнө; тэд бас хуваагаад цааш нь өгнө… Хүн бүр **ганц** хуудастай болоход эрэмбэлэх юм үлдэхгүй. Дараа нь эрэмбэлэгдсэн овоонуудыг хоёр хоёроор нь **нийлүүлсээр** буцаад нэг овоо болгоно.

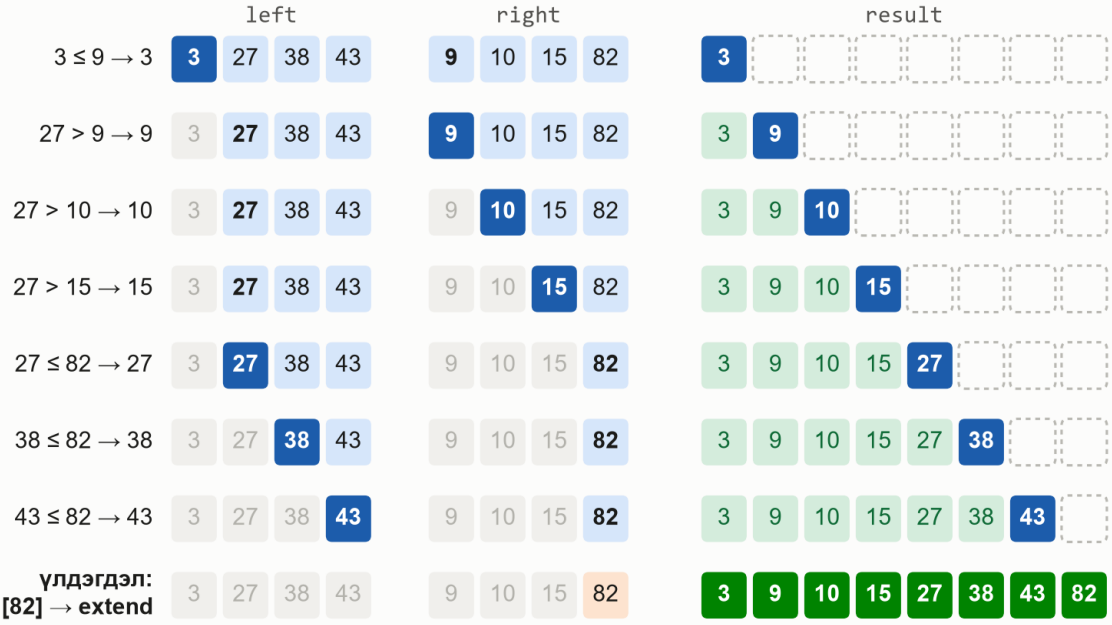
Энэ бол **хувааж ял** (divide and conquer) арга — гурван алхамтай:

1. **Хуваах:** жагсаалтыг дундуур нь хоёр хагас болгоно.
2. **Шийдэх:** хагас бүрийг **мөн merge sort-оор** эрэмбэлнэ (рекурс). 0 эсвэл 1 элементтэй жагсаалт аль хэдийн эрэмбэлэгдсэн — энэ бол **суурь тохиолдол**.
3. **Нийлүүлэх:** эрэмбэлэгдсэн хоёр хагасыг нэг эрэмбэлэгдсэн жагсаалт болгоно.

Бүх ажлын гол нь 3-р алхам. Тиймээс эхлээд нийлүүлэх функцээ бичье.

02 Нийлүүлэх (merge)

Эрэмбэлэгдсэн хоёр овоо хуудас ширээн дээр байна, хамгийн бага нь дээрээ. Шинэ овоонд дараагийн хуудсыг тавихын тулд **хоёр дээд** хуудсыг л харьцуулахад хангалттай — бага нь үлдсэн бүх хуудаснаас хамгийн бага нь гарцаагүй.



Зураг 1. merge([3, 27, 38, 43], [9, 10, 15, 82]). Мөр бүр нэг харьцуулалт: хар хөх — сонгогдсон бага нь (result-д нэмэгдэнэ), саарал — аль хэдийн авсан, улбар шар — үлдэгдэл. 7 харьцуулалтын дараа left дуусч, 82-ыг шууд залгав.

```python
def merge(left, right):
    result = []
    i = j = 0                                  # хоёр овооны «дээд хуудас»
    while i < len(left) and j < len(right):    # хоёуланд элемент үлдсэн үед
        if left[i] <= right[j]:                # бага нь → result
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])                    # аль нэг нь дуусав:
    result.extend(right[j:])                   # үлдсэнийг бүгдийг нь залгана
    return result

merge([3, 27, 38, 43], [9, 10, 15, 82])     # [3, 9, 10, 15, 27, 38, 43, 82]
```

Харьцуулалт бүр нэг элементийг result руу шилжүүлдэг тул нийт n элементтэй үед merge хамгийн ихдээ n − 1 харьцуулалт хийнэ — **O(n)**.

> **Яагаад < биш <= вэ?**
> Тэнцүү үед **зүүн** талынхыг түрүүлж авна. Зүүн хагас анхны жагсаалтад өмнө нь байсан тул ижил элементүүдийн дараалал хадгалагдана — merge sort **тогтвортой** (Эрэмбэлэлт 2/3-ийг сана).

> **Үлдэгдэл**
> Давталт аль нэг жагсаалт дуусахад зогсоно. Нөгөөд нь үлдсэн элементүүд эрэмбэлэгдсэн бөгөөд result-ийн бүх элементээс их тул шууд залгана. Хоёр extend-ийн нэг нь үргэлж хоосон жагсаалт залгадаг — энэ нь зүгээр.

03 Merge sort

Алхам 1. Суурь тохиолдол: 0 эсвэл 1 элемент бол эрэмбэлэх юм алга — хуулбарыг буцаана.

Алхам 2. Дундуур нь хуваана: `mid = len(items) // 2`

Алхам 3. Хагас бүрийг **өөрөөр нь** (рекурс) эрэмбэлнэ.

Алхам 4. Эрэмбэлэгдсэн хоёр хагасыг `merge`-ээр нийлүүлж буцаана.

```
def merge_sort(items):
    if len(items) <= 1:                # 1. суурь тохиолдол
        return items[:]                #    (хуулбар)
    mid = len(items) // 2              # 2. хуваана
    left = merge_sort(items[:mid])     # 3. зүүн хагасыг эрэмбэлнэ
    right = merge_sort(items[mid:])    #    баруун хагасыг эрэмбэлнэ
    return merge(left, right)          # 4. нийлүүлнэ

nums = [38, 27, 43, 3, 9, 82, 10, 15]
print(merge_sort(nums))     # [3, 9, 10, 15, 27, 38, 43, 82]
print(nums)                 # [38, 27, 43, 3, 9, 82, 10, 15] — өөрчлөгдөөгүй
```

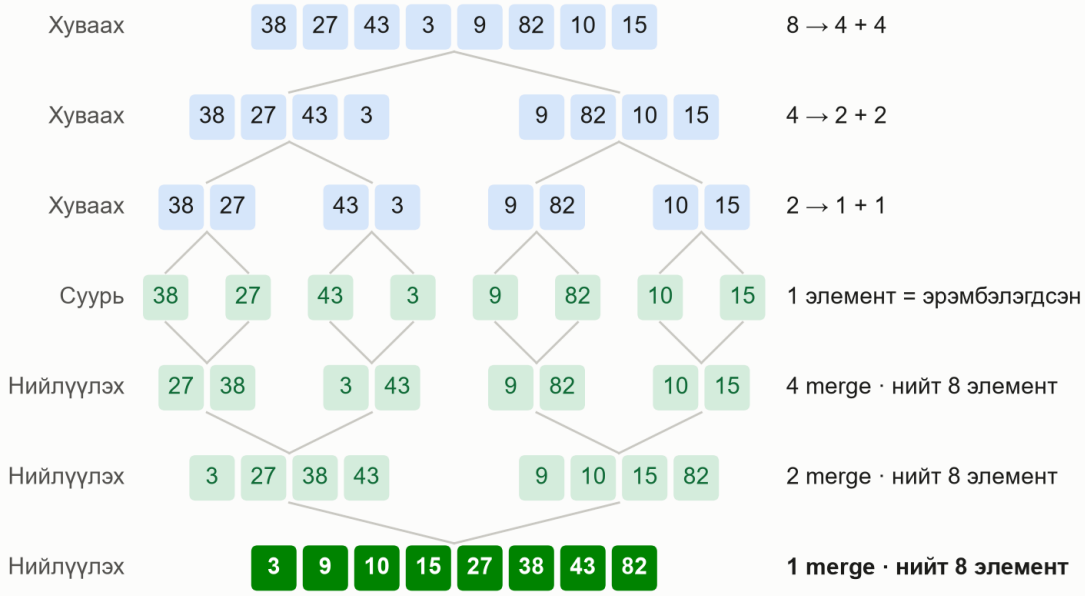
Байрандаа биш — шинэ жагсаалт

Selection, insertion sort `list.sort()` шиг байрандаа эрэмбэлдэг байсан. Энэ merge_sort нь `sorted()` шиг **шинэ** жагсаалт буцаадаг, эх жагсаалт хэвээр үлдэнэ. Тиймээс үр дүнг заавал `result = merge_sort(nums)` гэж хүлээж авна. Үнэ нь — O(n) нэмэлт санах ой.

Рекурс хэзээ зогсох вэ?

Дуудлага бүр жагсаалтыг хоёр хуваадаг тул урт нь 8 → 4 → 2 → 1 гэж багассаар суурь тохиолдолд **заавал** хүрнэ. Суурь тохиолдол буруу бол рекурс хэзээ ч зогсохгүй (6-р бүлэг).

04 Рекурсын мод



Зураг 2. merge_sort([38, 27, 43, 3, 9, 82, 10, 15])-ийн мод. Дээрээс доош — хуваах (merge_sort дуудлагууд), дооронос дээш — нийлүүлэх (merge). Цайвар хөх — эрэмбэлэгдээгүй, ногоон — эрэмбэлэгдсэн хэсэг.

Python модыг мөр мөрөөр биш, **зүүнээс эхэлж гүн рүү** гүйцэтгэдэг: эхлээд зүүн хагасыг бүрэн эрэмбэлж дуусаад дараа нь баруун хагас руу орно. Үүнийг харахын тулд дуудлага бүрийг гүнээр нь догол мөр болгон хэвлэе:

```
def merge_sort(items, depth=0):
    pad = "    " * depth                    # гүн бүрд 4 зай догол
    print(pad + f"merge_sort({items})")
    if len(items) <= 1:
        return items[:]
    mid = len(items) // 2
    left = merge_sort(items[:mid], depth + 1)
    right = merge_sort(items[mid:], depth + 1)
    result = merge(left, right)
    print(pad + f"merge → {result}")
    return result

merge_sort([38, 27, 43, 3])
```

Гаралт (бодитоор ажиллуулсан):

```
merge_sort([38, 27, 43, 3])
    merge_sort([38, 27])
        merge_sort([38])
        merge_sort([27])
    merge → [27, 38]
    merge_sort([43, 3])
        merge_sort([43])
        merge_sort([3])
    merge → [3, 43]
merge → [3, 27, 38, 43]
```

05 Яагаад O(n log n) вэ?

Модыг **түвшнээр** нь тоолъё (Зураг 2):

- **Түвшин бүрийн ажил ≈ n.** Нэг түвшний бүх merge нийлээд яг n элементийг нэг нэг удаа result руу хийнэ: 4 merge × 2 = 2 merge × 4 = 1 merge × 8 = 8 элемент.
- **Түвшний тоо ≈ $\log_2$ n.** Түвшин бүрт жагсаалт хоёр хуваагдана: 8 → 4 → 2 → 1 бол 3 түвшин. Binary search-ийнх шиг сая элементэд ердөө 20.
- **Нийт ажил ≈ n · $\log_2$ n → O(n log n)**. Хамгийн сайн, дундаж, хамгийн муу тохиолдол бүгд ижил.

n	Түвшин	merge sort ≈ n · $\log_2$ n	selection sort n(n − 1) / 2	Хэд дахин цөөн
10	4	33	45	≈ 1.4
1 000	10	9 966	499 500	≈ 50
1 000 000	20	19 931 569	≈ 5 · 10^{11}	≈ 25 086

Бодитоор тоолбол: 1 000 элементтэй санамсаргүй жагсаалтад merge_sort **8 711** харьцуулалт хийв, selection sort — **499 500**.

> **Үнэ: санах ой**
> Merge sort нэмэлт O(n) санах ой хэрэглэдэг (шинэ жагсаалтууд). Selection, insertion sort — O(1). Ихэнх тохиолдолд хурд нь илүү чухал.

06 Түгээмэл алдаа ба тест

Алдаа 1 · Үлдэгдлийг мартах

БУРУУ

```
while i < len(left) and j < len(right):
    ...
return result
```

ЗӨВ

```
while i < len(left) and j < len(right):
    ...
result.extend(left[i:])
result.extend(right[j:])
return result
```

Юу болох вэ: `merge([1, 5], [2, 3])` → `[1, 2, 3]` — 5 алга. `merge_sort([38, 27, 43, 3])` → `[3]`!

Яагаад: Давталт аль нэг жагсаалт дуусахад зогсдог. Нөгөөд нь үлдсэн элементүүдийг залгахгүй бол алдагдана. Түвшин бүрт алдагдсаар 4 элементээс 1 л үлдлээ.

Алдаа 2 · Суурь тохиолдол буруу

БУРУУ

```
if len(items) == 0:
    return []
```

ЗӨВ

```
if len(items) <= 1:
    return items[:]
```

Юу болох вэ: `merge_sort([5])` → `RecursionError`.

Яагаад: 1 элементтэй жагсаалтыг хуваахад mid = 0: зүүн хагас [], баруун хагас [5] — яг өмнөхтэйгөө ижил! Жагсаалт багасахгүй тул рекурс зогсохгүй, Python ≈ 1 000 гүнд алдаа шиднэ.

Алдаа 3 · Буцаасан утгыг хаях

БУРУУ

```
left, right = items[:mid], items[mid:]
merge_sort(left)
merge_sort(right)
return merge(left, right)
```

ЗӨВ

```
left = merge_sort(items[:mid])
right = merge_sort(items[mid:])
return merge(left, right)
```

Юу болох вэ: `merge_sort([38, 27, 43, 3])` → `[38, 27, 43, 3]` — эрэмбэлэгдээгүй.

Яагаад: merge_sort шинэ жагсаалт буцаадаг, left-ийг өөрийг нь өөрчилдөггүй. Үр дүнг хаясан тул merge эрэмбэлэгдээгүй хагасуудыг авч, «хоёр дээд хуудас» гэсэн таамаг эвдэрнэ.

Тест

Өмнөх хичээлүүдийн адил `sorted()`-тэй харьцуулна. Нэмээд эх жагсаалт **өөрчлөгдөөгүйг** шалгана:

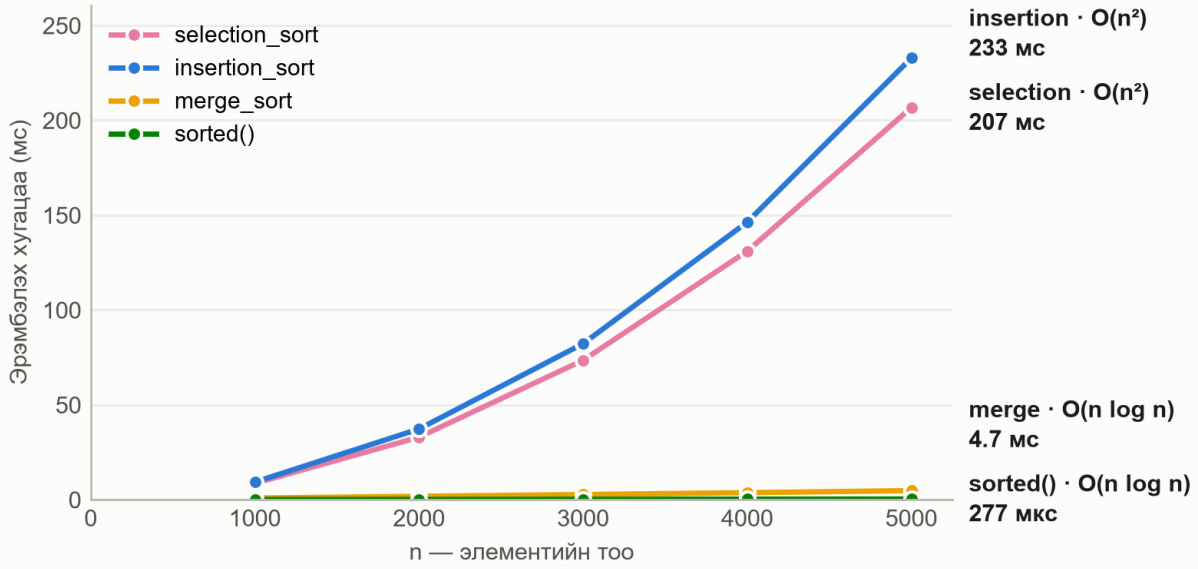
```
for _ in range(10_000):
    items = [random.randint(0, 9) for _ in range(random.randint(0, 12))]
    original = items[:]
    assert merge_sort(items) == sorted(items)
    assert items == original              # эх жагсаалт хэвээр
```

Зөв хувилбар **10 000** санамсаргүй тестийг бүгдийг давсан. Алдаатай хувилбарууд:

Хувилбар	1 000 тестээс бүтэлгүйтсэн	Хамгийн богино жишээ	Алдааны төрөл
Алдаа 1: үлдэгдэл мартсан	**834**	`[0, 0]` → `[0]`	буруу хариу
Алдаа 2: `len(items) == 0`	**922**	`[0]` → `RecursionError`	RecursionError
Алдаа 3: буцаасан утгыг хаясан	**663**	`[0, 7, 3]` → `[0, 7, 3]`	буруу хариу
Зөв `merge_sort`	**0**	—	—

07 Гурван алгоритмыг хэмжих

n = 1 000 … 5 000



Зураг 3. Санамсаргүй жагсаалтыг эрэмбэлэх хугацаа. Энэ PDF-ийг үүсгэх үед ижил өгөгдөл дээр бодитоор хэмжсэн. O(n²) хоёр алгоритм дээшээ муруйж, O(n log n) хоёр нь тэг орчимд хэвтэнэ.

Том n дээр

n	■ selection_sort	■ insertion_sort	■ merge_sort	■ sorted()
5 000	207 мс	233 мс	4.7 мс	277 мкс
100 000	≈ 1 мин	≈ 2 мин	**121 мс**	**9.0 мс**
1 000 000	≈ 2.3 цаг	≈ 2.6 цаг	**1.6 сек**	**123 мс**

≈ — O(n²) хоёрыг n = 5 000-ийн хэмжилтээс n²-аар үржүүлж **тооцоолсон** (тэд ингэж удаан ажиллах тул хэмжээгүй). Бусад нь бодит хэмжилт.

Сая элементэд merge sort **1.6 сек**, selection sort бол ойролцоогоор **2.3 цаг**. Энэ бол Big O хичээлийн гол санаа: **сайн алгоритм хурдан компьютерээс илүү чухал**. Харин `sorted()` нь merge sort-оос ч ≈ 13 дахин хурдан — C хэл дээр бичигдсэн, илүү ухаалаг алгоритм.

08 Python-ий sorted()

Ажлын кодонд өөрийн эрэмбэлэлтийг бараг бичихгүй — бэлэн функц хурдан, тогтвортой, алдаагүй. Харин яаж **зөв** хэрэглэхийг мэдэх хэрэгтэй:

```
nums = [38, 27, 43, 3, 9]
sorted(nums)                    # [3, 9, 27, 38, 43] — шинэ жагсаалт
sorted(nums, reverse=True)      # [43, 38, 27, 9, 3] — буурахаар
nums.sort()                     # байрандаа, None буцаана

words = ["банана", "алим", "үзэм", "лийр"]
sorted(words, key=len)          # үгийн уртаар: ['алим', 'үзэм', 'лийр', 'банана']

students = [("Бат", 85), ("Сараа", 92), ("Дорж", 85)]
sorted(students, key=lambda s: s[1], reverse=True)
# [('Сараа', 92), ('Бат', 85), ('Дорж', 85)] — тогтвортой: Бат Доржоос өмнө хэвээр
```

`key=` нь элемент бүрээс «юугаар нь харьцуулахыг» гаргаж өгөх функц. Ижил түлхүүртэй элементүүдийн анхны дараалал хадгалагдана (тогтвортой).

Анхаар: монгол үгийг эрэмбэлэх

`sorted()` үсгийг Unicode кодоор нь харьцуулдаг. Ө, Ү үсгийн код Я-гаас ч их тул цагаан толгойн дараалал алдагдана. Засахын тулд өөрийн key функц бичнэ:

```
MN = "абвгдеёжзийклмноөпрстуүфхцчшщъыьэюя"

def mn_key(word):
    return [MN.find(ch) for ch in word.lower()]   # үсэг бүрийн байр

words = ["үзэм", "тоор", "өвс", "алим"]
sorted(words)               # ['алим', 'тоор', 'үзэм', 'өвс']  ← буруу!
sorted(words, key=mn_key)   # ['алим', 'өвс', 'тоор', 'үзэм']
```

`sorted()` дотор юу байдаг вэ?

Timsort (Python 3.11-ээс хойш түүний сайжруулсан хувилбар) — merge sort ба insertion sort-ийн эрлийз. Жагсаалтаас аль хэдийн эрэмбэлэгдсэн хэсгүүдийг олж, богино хэсгүүдийг insertion sort-оор эрэмбэлээд, merge sort шиг нийлүүлдэг. Тиймээс хамгийн муу үед O(n log n), эрэмбэлэгдсэн өгөгдөлд O(n), тогтвортой — энэ цувралын 2/3, 3/3 хичээлийн санаанууд нийлсэн. Сая элементийг санамсаргүй бол **123 мс**, аль хэдийн эрэмбэлэгдсэн бол **4.2 мс**-д эрэмбэлэв.

09 Дасгал

Эхлээд өөрөө бодож, кодоо тестлээрэй. Хариуг дараагийн хуудаснаас шалга.

Дасгал 1 · merge-ийг мөрдөх

`merge([1, 4, 9], [2, 3, 10, 12])`: алхам бүрт аль элемент result руу орохыг бич. Нийт хэдэн харьцуулалт хийх вэ?

Дасгал 2 · Мод зурах

`merge_sort([5, 2, 4, 7, 1, 3, 2, 6])`-ийн модыг Зураг 2 шиг зур: хуваах ба нийлүүлэх түвшин бүрийг бич.

Дасгал 3 · Буурахаар

`merge_sort_desc(items)` бич. Тогтвортой хэвээр байлгахын тулд merge дахь харьцуулалтыг юу болгох вэ?

```
merge_sort_desc([3, 8, 1, 9, 4])    # [9, 8, 4, 3, 1]
```

Дасгал 4 · Түвшний тоо

16, 1 000, 1 000 000 элементтэй жагсаалтын рекурсын мод хэдэн нийлүүлэх түвшинтэй вэ?

Дасгал 5 · Нийтлэг элемент

`common(a, b)`: эрэмбэлэгдсэн (давхардалгүй) хоёр жагсаалтын **хоёуланд** байгаа элементүүдийг буцаа. merge-ийн «хоёр заагч» санааг ашиглаж O(n + m)-д бич.

```
common([1, 3, 4, 7, 9], [2, 3, 7, 8, 9])    # [3, 7, 9]
```

Дасгалын хариу

Бүх хариуг энэ PDF-ийг үүсгэх үед жишээн дээр нь ажиллуулж шалгасан.

1. merge-ийг мөрдөх — 5 харьцуулалт

1 ≤ 2 → 1; 4 > 2 → 2; 4 > 3 → 3; 4 ≤ 10 → 4; 9 ≤ 10 → 9; left дуусав → үлдэгдэл `[10, 12]`. Үр дүн: `[1, 2, 3, 4, 9, 10, 12]`.

2. Мод зурах

Хуваах	`[5, 2, 4, 7, 1, 3, 2, 6]`
Хуваах	`[5, 2, 4, 7] [1, 3, 2, 6]`
Хуваах	`[5, 2] [4, 7] [1, 3] [2, 6]`
Суурь	`[5] [2] [4] [7] [1] [3] [2] [6]`
Нийлүүлэх	`[2, 5] [4, 7] [1, 3] [2, 6]`
Нийлүүлэх	`[2, 4, 5, 7] [1, 2, 3, 6]`
Нийлүүлэх	`[1, 2, 2, 3, 4, 5, 6, 7]`

3. Буурахаар — >= хэрэглэнэ

```
def merge_desc(left, right):
    ...
        if left[i] >= right[j]:         # <=  →  >= : тэнцүү үед ЗҮҮНийг авна
    ...

def merge_sort_desc(items):
    if len(items) <= 1:
        return items[:]
    mid = len(items) // 2
    return merge_desc(merge_sort_desc(items[:mid]), merge_sort_desc(items[mid:]))
```

> бичвэл тэнцүү үед **баруун** талынх түрүүлж, тогтвортой байдал алдагдана.

4. Түвшний тоо — $\log_2 n$, дээш бүхэлчилсэн

16 → **4**, 1 000 → **10** (2^{10} = 1 024), 1 000 000 → **20** (2^{20} ≈ 1 сая). Binary search-ийн алхмын тоотой яг ижил.

5. Нийтлэг элемент — O(n + m)

```python
def common(a, b):
    result = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] == b[j]:                # хоёуланд байна
            result.append(a[i])
            i += 1
            j += 1
        elif a[i] < b[j]:               # бага нь нөгөөд байхгүй — алгасна
            i += 1
        else:
            j += 1
    return result
```

10 Хураангуй

Загвар — цээжлэх биш, ойлгож бич

■ **merge · O(n)**

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

■ **merge_sort · O(n log n)**

```
def merge_sort(items):
    if len(items) <= 1:
        return items[:]
    mid = len(items) // 2
    left = merge_sort(items[:mid])
    right = merge_sort(items[mid:])
    return merge(left, right)
```

Цувралын харьцуулалт

Алгоритм	Санаа	Сайн	Муу	Санах ой	Тогтвортой
■ Selection	хамгийн багыг сонгож урд тавина	O(n²)	O(n²)	O(1)	Үгүй
■ Insertion	дараагийнхыг зөв байранд оруулна	O(n)	O(n²)	O(1)	Тийм
■ Merge	хоёр хуваж, эрэмбэлж, нийлүүлнэ	O(n log n)	O(n log n)	O(n)	Тийм
■ sorted()	Timsort: merge + insertion	O(n)	O(n log n)	O(n)	Тийм

Санаж үлдэх зүйлс

- **Хуваж ял:** хуваа → хагас бүрийг рекурсээр шийд → нийлүүл. Суурь тохиолдол: ≤ 1 элемент.
- merge нь хоёр «дээд элемент»-ийг харьцуулна; дуусахад **үлдэгдлийг** залга; `<=` → тогтвортой.
- $\log_2 n$ түвшин × түвшин бүрт n ажил = **O(n log n)**; нэмэлт санах ой O(n).
- merge_sort шинэ жагсаалт буцаана — `result = merge_sort(nums)`.
- Ажлын кодонд `sorted()` / `list.sort()`; `key=`, `reverse=`-ийг ашигла; монгол үгэнд өөрийн key хэрэгтэй.